ISA SPECIFICATION

---

1. Overview & Motivation

- **THE PURPOSE AND APPLICATION CONTEXT OF OUR ISA.**
  The purpose of this custom ISA is to serve as the core of a low-cost, AI-enabled mobile processor designed for the Lesotho market. The target application context is *Voice and Language Processing for Local Accessibility*, specifically supporting workloads like Sesotho voice dictation, voice biometric security, and translation/read-aloud assistants. These applications are critical for overcoming literacy, language, and accessibility barriers in digital services.
- **THE DESIGN GOALS.**
  - **High Efficiency on DSP[1]/AI Workloads:** Optimize for the Multiply-Accumulate (MAC) operations that dominate the target applications (e.g., FFT, MFCC[2], dot products, similarity checks).
  - **Real-Time Performance:** Ensure low-latency processing of audio streams to enable responsive user interactions.
  - **Low Power Consumption:** Suit the constraints of mobile devices (High performance on long lasting Battery).
  - **Hardware Simplicity:** Keep the design simple enough to be feasible for a low-cost processor while meeting performance goals.
- **WHY THIS ISA DESIGN IS SUITABLE FOR THE TARGET WORKLOAD.**
  The analysis of representative applications (VoiceDictation.cpp, VoiceBiometricSecurity.cpp, Translation.cpp) reveals a common pattern: intensive vector and matrix arithmetic, particularly dot products and trigonometric functions (e.g., sin in Translation.cpp). Our ISA directly addresses this by including native support for SIMD (Single Instruction, Multiple Data) operations and a dedicated MAC instruction, reducing the instruction count and cycle time for these core computational tasks.

[1] DSA refers to Digital Signal Processing.

[2] MFCC (Mel-Frequency Cepstral Coefficients) is a feature extraction technique used to convert raw audio into a compact representation that's optimal for voice recognition.

2. Architectural Design Choices

- **INSTRUCTION PHILOSOPHY**:
  RISC-style. A Reduced Instruction Set Computer philosophy is chosen for its simplicity, which leads to a cleaner pipeline design, higher clock frequencies, and lower power consumption—all critical for our target mobile device. The complexity of the workloads is handled by a small set of powerful, specialized instructions (like SIMD MAC) rather than a large set of complex ones.
- **REGISTERS**:
  - **Number:** 32 general-purpose registers (GPRs), x0-x31.
  - **Size:** 32 bits. This is sufficient for the 16-bit audio samples and 32-bit floating-point values common in our workloads.
  - **Roles:**
    - x0: Hardwired to zero.
    - x1: Return address for procedure calls.
    - X2: Stack pointer.
    - x5- x7: Used for linking and temporary values in our custom instructions.
    - x8- x15: Argument/result registers.
    - x16- x31: General-purpose temporary registers.
- **DATA TYPES**:
  The ISA natively supports 8-bit (byte), 16-bit (half-word), and 32-bit (word) integer data types. It also supports 32-bit single-precision floating-point, which is essential for audio processing and similarity calculations.
- **ADDRESSING**:
  - **Register Direct:** SUM rd, rs1, rs2
  - **Immediate:** SUMI rd, rs1, imm12
  - **Base + Offset (Displacement):** GET rd, imm12(rs1) (for memory load/store)
  - **PC-Relative:** JUMPL rd, imm20 (for jumps and function calls)
- **MEMORY MODEL**:
  Little-endian byte ordering. Memory accesses are required to be **naturally aligned** (words on 4-byte boundaries, half-words on 2-byte boundaries) to simplify the hardware design. Misaligned accesses will cause a trap. Example, vectors are word aligned (addresses divisible by 4).
- **INSTRUCTION FORMATS**:
  Fixed-length 32-bit instruction format. This simplifies instruction fetch and decode. We use a format similar to RISC-V for familiarity and efficiency:
  - **R-type:** Register-Register operations (e.g., SUM, VMAC).
  - **I-type:** Register-Immediate operations and loads (e.g., SUMI, GETW).
  - **S-type:** Store operations (e.g., PUTW).
  - **B-type:** Conditional branches (e.g., BRANCHEQ).
  - **U-type:** Upper Immediate instructions (e.g., LUI).
  - **J-type:** Jump instructions (e.g., JUMPL).

3. Instruction Set Summary

Group	Mnemonic	Operands	Description
Arithmetic	SUM	rd, rs1, rs2	Integer Add
	SUMI	rd, rs1, imm12	Integer Add Immediate
	DIFF	rd, rs1, rs2	Integer Subtract
	PROD	rd, rs1, rs2	Integer Multiply (Lows)
Logical	AND, OR, XOR	rd, rs1, rs2	Bitwise Logical Operations
	ANDI, ORI, XORI	rd, rs1, imm12	Bitwise Logical Ops (Immediate)
Memory	GETW	rd, imm12(rs1)	Load Word
	PUTW	rs2, imm12(rs1)	Store Word
	GETH/GETB/PUTH/PUTB	...	Load/Store Half-word/Byte
Control	BRANCHEQ,BRANCHNQ	rs1, rs2, imm12	Branch if Equal / Not Equal
	JUMPL	rd, imm20	Jump and Link
	JUMPLR	rd, rs1, imm12	Jump and Link Register

Group	Mnemonic	Operands	Description
DSP/Vector	VMAC	rd, rs1, rs2	**Vector Multiply-Accumulate**: rd = rd + (rs1 * rss2)
	VSIMD_ADD	rd, rs1, rs2	**SIMD Add** (4x 8-bit elements packed in 32-bit register)
	VSIMD_MUL	rd, rs1, rs2	**SIMD Multiply** (4x 8-bit elements)
	VHSUM	rd, rs1,	Horizontal sum of vector elements. (rs1 = [a, b, c, d] , rd = a+b+c+d)
MATH	FROOT	rd, rs1	Floating-Point Square Root (for `norm` calculation)
	FSIN	rd, rs1	Floating-Point Sine (for waveform synthesis)

---

4. Instruction Encoding Summary

The primary instruction format is a fixed 32-bit length. The bit fields for the core R-type format are as follows:

Bits 31-25	Bits 24-20	Bits 19-15	Bits 14-12	Bits 11-7	Bits 6-0
funct7	rs2	rs2	Funct3	rd	opcode

Field Sizes:

- **opcode:** 7 bits (identifies the base instruction type)
- **rd, rs1, rs2:** 5 bits each (addressing the 32 registers)
- **funct3:** 3 bits (specifies operation variant, e.g., ADD vs. SUB)
- **funct7:** 7 bits (used to extend the opcode for specialized instructions like VMAC)

The primary instruction format for unary instructions like FSIN, FSQRT AND VHSUM is an I-type and is as follows:

Bits 31-20	Bits 19-15	Bits 14-12	Bits 11-7	Bits 6-0
Imm[11-0]	rs2	Funct3	rd	opcode

Field Sizes:

- **opcode:** 7 bits (identifies the base instruction type)
- **rd, rs1:** 5 bits each (addressing the 32 registers)
- **funct3**: Used to distinguish between different unary operations
- **imm [11:0]**: Set to 0 (or used for function variants)

Example:

Imm [11:0] = 000000000000 (12 bits of zeros)

rs1 = source register (e.g., 01010 for x10)

funct3 = 001 (could mean "sine" operation)

rd = destination register (e.g., 01011 for x11)

opcode = 1011011 (custom opcode space)

Example Encoding Pattern:

- Standard Integer R-type instructions (SUM, DIFF, etc.) use funct3 and funct7 to differentiate.
- Our custom VMAC instruction reuse the R-type format but is assigned a unique opcode in the "Custom" RISC-V space (e.g., opcode 1011011). The funct7 field is set to a specific value to distinguish it from other future custom instructions.
- Immediate-based formats (I-type, S-type) repurpose the rs2 and funct7 fields to form a larger immediate value. Used also to represent customized unary instructions, where the imm field is set to all 0`s.

---

5. Design Rationale & Trade-offs

- **SIMPLICITY VS CAPABILITY:**
  We prioritized capability for our specific domain without overly complicating the design. We included VMAC, FROOT, and FSIN because they are performance-critical bottlenecks in our applications (e.g., vector_dot, norm, synthesize). Excluding them would result in long, inefficient software routines. We excluded more complex features like out-of-order execution or large vector units to maintain hardware simplicity and low power.
- **CODE DENSITY VS PERFORMANCE:**
  The fixed-length 32-bit format is less compact than a variable-length one but simplifies decoding. The performance gains from specialized instructions like VMAC far outweigh the cost of lower code density for our core loops. A single VMAC can replace three standard instructions (PROD, SUM, GET/PUT for loop index), significantly improving performance and reducing code size in critical sections.
- **HARDWARE IMPACT:**
  The RISC core simplifies the datapath. The custom instructions complicate it moderately:
  - VMAC requires an additional pipeline stage or a dedicated functional unit connected to the register file.
  - FROOT and FSIN require integrating a Floating-Point Unit (FPU), which adds area and power but is essential.

  The control logic remains relatively simple due to the regular instruction format.
- **EXTENSIBILITY:**
  Yes. The instruction encoding reserves space in the funct7 and opcode fields for new custom instructions. The 32-bit fixed format provides a stable base. Future extensions could include more SIMD widths, support for 64-bit data, or instructions for other AI kernels like convolution.